Chair of Transportation Systems Engineering
TUM School of Engineering and Design
Technical University of Munich

Statistical Learning and Data Analytics for Transportation Systems
M.Sc. Transportation Systems

Problem Set 3

Topics: Neural Networks, Reinforcement Learning, Convolutions
Due date: 31.08.2026, 23:59
Grading: 30% of your final grade

Notes:

- **Report submission**: Submit your answers/reports to **Moodle** before the deadline. Submissions via email will NOT be accepted. Only the reports submitted to Moodle will be evaluated.
- **Report format**: You need to submit a zip file containing: (1) a report as a PDF document that contains your solutions to all problems, and (2) a supplementary Jupyter notebook (`.ipynb`) or similar code files.

  The zip file should be named as "`Matriculation number, Name`". For example, "`03709721, Max Mustermann.zip`". This information should also be provided at the beginning of the report and the notebook/code.

  Note that the notebook/code should be submitted with the outputs (i.e., after running each cell). You can use any programming language you like.
- **Report evaluation**: You are evaluated based on your reports and the final answers. So, you have to describe and explain the steps to solve each problem as clearly as possible.
- ⚠ You need to solve these problems individually, and all text needs to be written in your own words. Solutions will be checked for plagiarism and originality. Copying others' solutions will NOT be tolerated and will result in a failing grade in this course.
- ⚠ Late submission will NOT be accepted. If you encounter any technical issues, please contact the corresponding lecturer.
- ⚠ Handwritten answers are NOT accepted. All submissions must be typed digitally.

1. Problem 1 (15 points)

An incomplete implementation of the Multilayer Perceptron (MLP) model is provided in `mlp.py`. You are expected to complete the missing parts of the code. The optimiser is set to the simple full-batch gradient descent with a fixed learning rate.

A testing snippet is also provided to help you validate your implementation. The input feature matrix is a 2D array with shape (`n_samples`, n_x), and the output label is a 2D array with shape (`n_samples`, 1).

*Please include your **code** for Problem 1. Complete Jupyter Notebooks (`.ipynb`) or equivalent script outputs must be submitted alongside the report.*

1. Derive the gradient of the loss function with respect to the weights and biases. Complete the implementation of the MLP model in `mlp.py`, and show the plot of classification results on the test data. (8 points)

2. Identify the best hyperparameters `learning_rate` and `alpha` using Bayesian optimization. (7 points)

2. Problem 2 (25 points)

You are asked to analyse the demand pattern of taxi rides to help improve the fleet management efficiency. The dataset `nyc-yellow-may2oct.csv` contains the demand data from May to October[1]. A detailed description of the variables is given in Table 1. You are expected to develop a deep learning model to forecast pickup and drop-off demand over the next four time steps.

Table 1: Variable description for Problem 2 data

Variable code	Description
`datetime`	Date time (15-min interval)
`location_id`	Location ID
`pickup_demand`	Total number of pickups / 15 min
`dropoff_demand`	Total number of dropoffs / 15 min

*Please include your **code** for Problem 2.*

1. How to cleanse and prepare the dataset in a format that can be used for training your model? Write a Dataset class to provide preprocessed input features and output labels. Then split the dataset into training, validation, and test subsets. (5 points)

2. Which types of neural networks are the most suitable for solving this problem? Elaborate on your architecture design. Train your model with appropriate settings and evaluate its performance using proper metrics. (15 points)

3. Discuss the theoretical advantages and potential drawbacks of using a deep learning approach compared to traditional statistical time-series methods (such as ARIMA or Historical Average) for this specific taxi demand forecasting task. (5 points)

[1]Data source: `https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page`

3. Problem 3 (30 points)

Emily is a green-conscious citizen of Electropolis who loves her sleek electric vehicle. Today, however, Emily finds herself in a predicament. Her car's battery is dangerously low, and the dashboard indicator is flashing a red warning. Therefore, she must navigate the local road network to find a charging station as soon as possible.

The road network can be modelled as a 3×3 grid of 9 nodes, where each of the 9 nodes represents a state. As shown in Figure 1, adjacent nodes are connected by bidirectional roads (edges), with different colours indicating varying congestion levels[2].

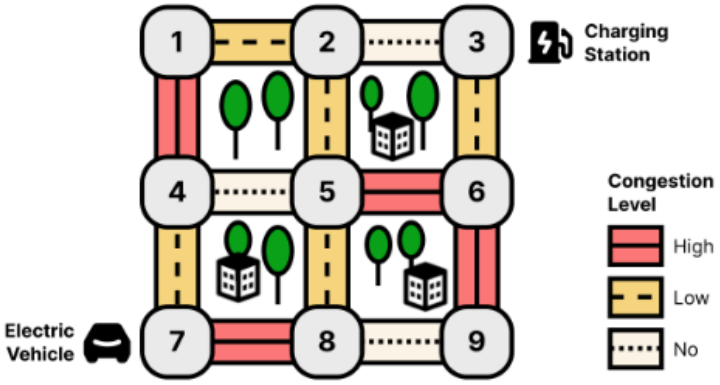

Figure 1: Road network

The nearest charging station with vacant slots is located at Node 3. In each time step, Emily must move to an adjacent node; she CANNOT remain stationary. Each move yields a reward determined by the road's congestion level, as detailed in Table 2. Additionally, she will receive a bonus of $+10$ upon successfully reaching the charging station.

Table 2: Reward list

Congestion Level	Reward
No congestion	-1
Low congestion	-3
High congestion	-5

1. Consider the route $7 \to 8 \to 5 \to 2 \to 3$. Given a discount factor of $\gamma = 0.9$, calculate the total discounted return. (6 points)

2. Initially, with no idea of which route is better, Emily takes a uniform random policy. Show the initial policy and corresponding transition probabilities when Emily's state is at Node 5, Node 6, and Node 9, respectively. (5 points)

3. Starting from values $v(s) = 0$ for all states, perform two iterations of undiscounted policy evaluation, which means that the discount factor $\gamma = 1.0$. Show the updated values of all the states in each iteration. (12 points)

[2]Here we assume that the congestion levels of two directions on a road are the same.

4. Based on the updated state values in subproblem 3, determine the new greedy policy. Illustrate this new policy by drawing arrows on the grid. Is it the optimal policy? Discuss whether we can guarantee the optimality of an optimal solution obtained by policy iteration in real-world scenarios. (7 points)

4. Problem 4 (30 points)

Convolution is the core operation in Convolutional Neural Networks (CNNs). In this problem, you will explore how 2D convolution works by manually computing it, implementing the operation from scratch, and applying standard image-processing filters to extract features.

*Please include your **code** for Problem 6.2 and 6.3. Complete Jupyter Notebooks (`.ipynb`) or equivalent script outputs must be submitted alongside the report. **Do NOT** use built-in 2D convolution functions (e.g., `scipy.signal.convolve2d`, `cv2.filter2D`, or PyTorch/TensorFlow `conv2d` layers) for your implementation.*

1. **Manual Convolution:** Consider a 4×4 input grayscale image I and a 3×3 filter K designed for edge detection, given below:

$$I = \begin{bmatrix} 10 & 10 & 0 & 0 \\ 10 & 10 & 0 & 0 \\ 10 & 10 & 0 & 0 \\ 10 & 10 & 0 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

   Compute the 2D convolution $I * K$ assuming a stride of 1 and no padding (valid padding). Show the resulting output matrix and interpret what the filter has detected based on the values in image I. (5 points)

2. **Convolution from Scratch:** Write a Python function `conv2d(image, kernel, stride=1, padding=0)` that implements the 2D convolution operation using only basic NumPy (or equivalent) array operations (loops are permitted but vectorisation is encouraged). Your function should be able to handle a 2D grayscale image array, a 2D kernel array, and arbitrary `stride` and `padding` values. (12 points)

3. **Applying Image Filters:** Load any standard RGB image (you can use `skimage.data.camera()`, an image from `matplotlib`, or upload your favourite anime character), and convert it to grayscale. Using your custom `conv2d` function from the previous step, apply the following two edge-detection filters to the image:

   - **Sobel X (Horizontal derivative):** $K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$
   - **Sobel Y (Vertical derivative):** $K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$

Plot the original grayscale image and the two resulting feature maps side by side. Briefly comment on the visual differences between the outputs of K_x and K_y. It is suggested to use `stride` and `padding` both equal to 1, but feel free to explore other values to see the effects. (8 points)

4. **Connection to CNNs:** In this exercise, you used fixed, predefined weights for the Sobel filters to extract specific features (edges). Explain how this differs from the convolutional layers in a deep Convolutional Neural Network. How does a CNN determine the values inside its kernels, and why is this advantageous for complex tasks like image classification? (5 points)